

Actividad de Investigación y Práctica: Estructuras de Datos Avanzadas y APIs con ASP.NET Core

Parte 1: Investigación Teórica

El objetivo del presente documento es consolidar los fundamentos conceptuales de la organización de datos en la memoria volátil y su posterior exposición estructurada a través de servicios web modernos basados en el protocolo de transferencia de hipertexto.

1. Estructuras de Datos Eficientes

Árboles Binarios de Búsqueda (ABB)

Un Árbol Binario de Búsqueda es una estructura de datos jerárquica y dinámica de tipo no lineal, compuesta por nodos conectados entre sí donde existe un único nodo raíz principal.

- **Regla de ordenamiento:** Para cualquier nodo intermedio o raíz dado XX , todos los elementos y claves ubicados en su **subárbol izquierdo** deben poseer obligatoriamente un valor **menor** que XX , mientras que todos los elementos ubicados en su **subárbol derecho** deben poseer un valor estrictamente **mayor** que XX .
- **Desventaja principal (Degeneración en lista vinculada):** Si los datos son insertados en el árbol de manera ordenada o secuencial (por ejemplo, de forma estrictamente ascendente: 1, 2, 3, 4, 5), la estructura pierde por completo su capacidad de ramificación simétrica. Al agregarse cada nuevo nodo siempre al extremo derecho del nodo anterior, el árbol se deforma visual y operativamente en una **lista enlazada simple**. Como consecuencia directa de esta degeneración, la eficiencia del algoritmo de búsqueda se degrada críticamente, pasando de un escenario óptimo de $O(\log n)$ a un costo computacional lineal de $O(n)$ en el peor de los casos, anulando las ventajas de usar un árbol.

Árboles AVL

- **Definición de Árbol Auto-balanceado:** Es una variante optimizada de un árbol binario de búsqueda que implementa un mecanismo

automático de reestructuración física. Ante cualquier operación de escritura (inserción o eliminación de datos), el árbol evalúa su simetría y ejecuta de forma autónoma algoritmos de reacomodo llamados "rotaciones" (simples o dobles) para asegurar que la altura general permanezca en su mínimo posible.

- **Factor de Balanceo:** Es el indicador métrico fundamental calculado individualmente para cada nodo del árbol. Se define formalmente mediante la diferencia matemática de las alturas de sus respectivos subárboles adyacentes:

$$Factor = Altura_{\{Izquierda\}} - Altura_{\{Derecha\}}$$

Para preservar la propiedad de balanceo AVL, el factor resultante en cada nodo individual debe pertenecer estrictamente al conjunto de valores enteros $\{-1, 0, 1\}$. Si una mutación de datos (inserción o borrado) provoca que el factor de un nodo llegue a ± 2 , la estructura se declara en desbalance y desencadena un proceso inmediato de rotación para corregirse.

- **Complejidad constante $O(\log n)$:** Al garantizar de forma matemática estricta que la altura máxima (h) del árbol esté fuertemente acotada y sea directamente proporcional al logaritmo natural de sus elementos ($h \approx \log_2 n$), la distancia desde la raíz hasta la hoja más profunda es mínima. De este modo, la complejidad temporal de las operaciones críticas de búsqueda, inserción y eliminación se conserva rígidamente en el peor de los casos en un orden logarítmico $O(\log n)$.

2. Fundamentos de Web APIs

¿Qué es una API y cómo funciona el modelo Cliente-Servidor?

Una **API (Application Programming Interface)** actúa como un contrato formal y canal de comunicación estandarizado que faculta a dos sistemas de software formalmente independientes a intercambiar datos y ejecutar instrucciones mutuas de manera segura y transparente.

En el marco arquitectónico del modelo **Cliente-Servidor**, la lógica operativa se divide de forma explícita entre un solicitante de servicios (Cliente) y un proveedor centralizado de lógica o datos (Servidor o Web API):

- **El flujo de la Petición (Request):** El cliente (un navegador web, una aplicación móvil o una herramienta de pruebas como Postman) emite un mensaje estructurado que viaja a través de la infraestructura de red por medio del protocolo HTTP. Esta estructura incluye una dirección de destino unívoca (URI/URL), un método o verbo de acción, metadatos estructurados en cabeceras (*headers* con tipos de contenido o tokens de seguridad) y, en escenarios de creación o actualización, un cuerpo de datos (*body*) típicamente formateado bajo el estándar JSON.
- **El flujo de la Respuesta (Response):** El servidor recibe la trama de datos, procesa la intención del cliente (interactuando opcionalmente con motores de bases de datos mediante frameworks como ASP.NET Core) y despacha un mensaje de retorno. Dicha respuesta viaja de vuelta al solicitante provista de un **código de estado HTTP** numérico estandarizado (por ejemplo, 200 OK para operaciones exitosas, o 400 Bad Request ante inconsistencias), cabeceras de respuesta y la carga útil o recurso final (un objeto JSON con la información procesada).

Verbos HTTP: Análisis Conceptual de GET y POST

Característica	Verbo GET	Verbo POST
Propósito Conceptual	Exclusivo para la recuperación y lectura de recursos ya existentes en el servidor. Bajo ninguna circunstancia debe alterar o modificar el estado del sistema.	Destinado específicamente para la creación, procesamiento y persistencia de nuevos recursos en el repositorio del servidor.
Uso Correcto	Solicitud pura de datos del sistema (ej: consultar catálogos de productos,	Envío seguro y estructurado de datos transaccionales (ej: formularios de registro,

Característica	Verbo GET	Verbo POST
	perfiles o listados). Los argumentos de filtrado o paginación se exponen explícitamente en la URL de forma pública (<i>Query Strings</i>).	credenciales o compras). Los datos sensibles o extensos se encapsulan estrictamente dentro del cuerpo de la petición (<i>Body</i>).
Idempotencia	Sí posee Idempotencia. La ejecución idéntica e indefinida de una misma consulta GET generará siempre el mismo estado en el servidor sin provocar efectos colaterales ni duplicidad en los datos.	No posee Idempotencia. Si una misma petición de creación POST es despachada en repetidas ocasiones por error de red o reintento, el servidor interpretará cada envío como un evento independiente, generando registros múltiples y duplicados en la base de datos.